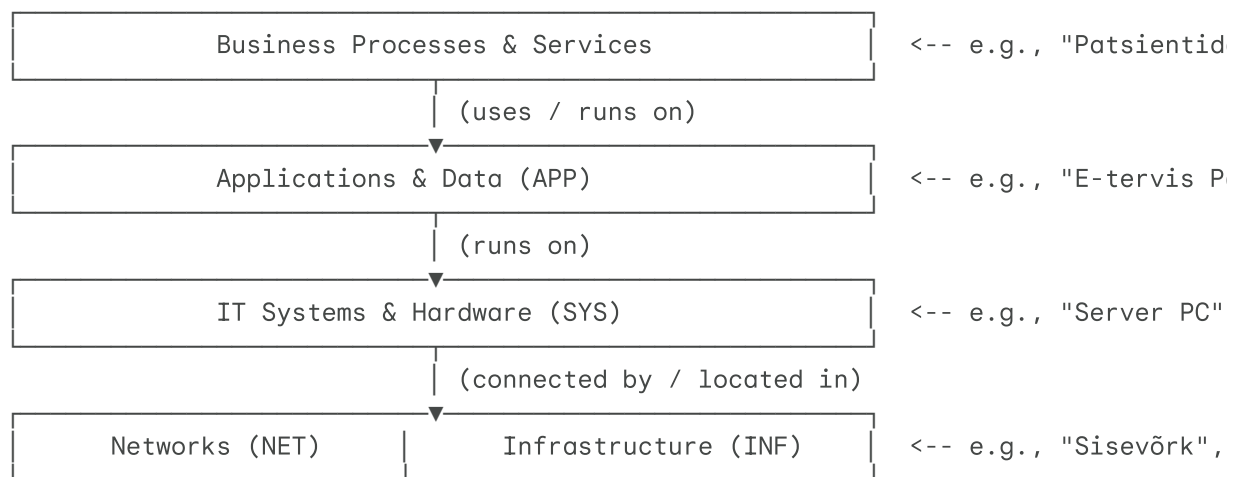


E-ITS Scope Modeling & Protection Needs Inheritance Guide

This guide outlines how to implement **Scope Modeling (Kaitseala modelleerimine)** and recursive **Protection Needs Inheritance (Kaitsetarbe pärilikkus)** within your E-ITS compliance application. It explains the conceptual layers, mathematical propagation of security levels, and how to represent these relationships in your UI and backend.

1. Conceptual Framework: The Layered Architecture

In E-ITS (based on BSI IT-Grundschutz), an organization's scope (kaitseala) is not a flat list of assets. It is a structured graph consisting of five layers. Assets in higher layers depend on assets in the lower layers to function:



Typical Relations to Model:

- Process** → **Asset**: A Business Process directly uses an application (APP) or a specific computer (SYS).
- App** → **System**: An Application (APP) runs on a virtual or physical Server (SYS).
- System** → **Infrastructure**: A physical Server (SYS) is physically located inside a Server Room or Office (INF).
- System** → **Network**: A computer (SYS) is connected to a specific Local Area Network (NET).

2. Mathematical Model of Protection Needs Propagation

In E-ITS, protection needs for Confidentiality (*C*), Integrity (*I*), and Availability (*A*) are evaluated using three discrete levels:

- Normal (N)** (Normaalne)
- High (S)** (Suur)
- Very High (V)** (Väga suur)

$$\text{Level}(x) \in \{1, 2, 3\} \quad \text{where} \quad N = 1, S = 2, V = 3$$

Let:

- $\mathcal{P}$ be the set of all Business Processes. Each process $p \in \mathcal{P}$ has an inherent protection need vector:

$$\mathbf{S}(p) = (C_p, I_p, A_p)$$

- $\mathcal{A}$ be the set of all Assets. Each asset $a \in \mathcal{A}$ has a baseline protection need vector assigned to it directly:

$$\mathbf{S}_{\text{baseline}}(a) = (C_{\text{baseline},a}, I_{\text{baseline},a}, A_{\text{baseline},a})$$

- $\mathcal{BP}(a) \subseteq \mathcal{P}$ be the set of business processes directly linked to asset a .
- $\mathcal{D}(a) \subseteq \mathcal{A}$ be the set of assets that directly depend on a (i.e., there exists a relation $a' \rightarrow a$, meaning a' runs on or is located inside a).

The Propagation Formulas (The Maximum Principle)

The inherited protection need $\mathbf{S}_{\text{inherited}}(a) = (C_a, I_a, A_a)$ for any asset a is calculated recursively as the maximum value of its own baseline, the processes using it, and all assets depending on it:

$$C_a = \max(\{C_p \mid p \in \mathcal{BP}(a)\} \cup \{C_{a'} \mid a' \in \mathcal{D}(a)\} \cup \{C_{\text{baseline},a}\})$$

$$I_a = \max(\{I_p \mid p \in \mathcal{BP}(a)\} \cup \{I_{a'} \mid a' \in \mathcal{D}(a)\} \cup \{I_{\text{baseline},a}\})$$

$$A_a = \max(\{A_p \mid p \in \mathcal{BP}(a)\} \cup \{A_{a'} \mid a' \in \mathcal{D}(a)\} \cup \{A_{\text{baseline},a}\})$$

This ensures that if a highly critical medical record application ($C = \text{Very High}$) is stored on a server in the basement, the server and the physical server room automatically inherit the $C = \text{Very High}$ requirement.

3. Database Schema Implementation

To support this in your Postgres database, we introduce two tables and a recursive view:

1. `business_process_assets` : Connects Business Processes to Assets.
2. `asset_relations` : Connects Assets to other Assets with defined relationship types (e.g., 'runs_on', 'located_in').
3. `v_asset_protection_inheritance` : A recursive Common Table Expression (CTE) view that automatically executes the formulas above.

Additionally, a database trigger checks for **circular dependencies** (e.g., Asset A depends on Asset B, which depends on Asset A) and rejects them to prevent infinite recursive loops.

4. UI/UX Integration Recommendations

A. The Dependency Tree Visualizer

Render a nested tree or node-graph showing where an asset sits in the hierarchy:

- Show that **Room 102 (INF)** contains **Client PC (SYS)**, which runs **E-ID Client (APP)**.
- Clearly highlight which assets are inheriting their protection needs. For example, if Room 102 is marked "High Protection Need", show a tool-tip: *"Inherited from Client PC → E-ID Client → Process: Patient Registration"*.

B. Prevention of Manual Mistakes

- When a user links an Asset to a Room, the application should restrict target options based on type (e.g., a physical server `SYS` can only be `located_in` an infrastructure asset `INF`).
- Always display the **Inherited Protection Need** side-by-side with the **Baseline Protection Need** so security officers can easily see why a low-priority asset has been upgraded to "Very High".